

ATLAS Threshold Detection Rules Analysis

Detection Rule Calibration via FRR / FAR / EER

AUTHOR
Patrick Leffler

PUBLISHED
July 16, 2026

ABSTRACT

This project uses the ATLAS (Alert Threshold Lifecycle Assessment System) method to assess whether three detection rules—Brute Force Login, Lateral Movement, and Data Exfiltration—are set at the appropriate thresholds based on their current confusion matrix data. For each rule, the analysis calculates the False Rejection Rate (FRR), False Acceptance Rate (FAR), and Equal Error Rate (EER), and then provides a clear TIGHTEN, LOOSEN, or HOLD recommendation based on the most common error type. Brute Force Login is causing too many false alarms compared to real threats and should be tightened. Lateral Movement is missing almost half of confirmed events and should be loosened, even though this will mean more work for analysts. Data Exfiltration is almost exactly at its EER and should stay at its current threshold. Beyond these findings, the report argues that setting thresholds is really about deciding how much risk to accept, not just a technical choice. If a rule is not calibrated, it means the risk tolerance is not clearly stated. Too many alerts from a high-FAR rule can lead to extra staffing costs, and regulators and auditors now expect to see records showing that detection controls are actually tested, not just installed. The final section explains this point for risk stakeholders and board members.

Overview

This report uses the ATLAS (Alert Threshold Lifecycle Assessment System) method from Data-Driven Cybersecurity (Mattei, Manning 2025) to check if detection thresholds are set correctly for each rule. Getting these thresholds right is key to making detection programs work well every day. If a rule is too strict, real intrusions can slip by unnoticed. If it is too loose, the system gets flooded with alerts, making it hard to spot real threats. ATLAS helps make these decisions based on clear, repeatable data instead of guesswork or recent high-profile incidents.

For each detection rule, this analysis looks at three metrics from the confusion matrix at the current threshold: False Rejection Rate (FRR), False Acceptance Rate (FAR), and Equal Error Rate (EER). Together, these numbers show where a rule stands between missing threats and raising false alarms, and whether that balance makes sense for the rule’s purpose. Based on the relationship between FRR and FAR, the report gives a clear recommendation—**TIGHTEN, LOOSEN, or HOLD**—so security engineers get practical advice they can use right away, not just more data to store.

The three detection rules reviewed here—Brute Force Login, Lateral Movement, and Data Exfiltration—were picked because they cover different stages of the attack process and have different risks if not set correctly. Missing a lateral movement alert leads to different problems than getting too many brute force alerts. The recommendations in this report take these differences into account, instead of treating all three rules the same.

ATLAS Methodology

ATLAS is an 8-step cyclical process: Select → Define → Categorize → Collect → Analyze → Implement → Measure → Report. Each step builds on the last: rules are selected for review, success criteria are defined, incoming events are categorized against ground truth, and data is collected over a fixed observation window before analysis begins. This report covers the **Analyze** and **Implement** steps — computing FRR, FAR, and EER from the collected confusion matrix, and translating that math into a specific threshold action for each rule. Re-running after threshold adjustment closes the **Measure** loop, and the resulting delta becomes input to the next **Report** step, which make

Methodology

FRR — False Rejection Rate

The FRR measures the proportion of actual threats that the detection rule failed to flag. A high FRR means the threshold is too strict: the rule is blocking legitimate signals and letting real threats through undetected. Because FRR is calculated only against the population of confirmed threats (true positives plus false negatives), it isolates a rule’s blind spot from its noise problem — a rule can have an excellent FAR and still be dangerously permissive toward actual attackers if its FRR is high. Tracking FRR on its own is what keeps an analyst from mistaking a quiet alert queue for a secure environment; the silence could just as easily mean the rule stopped seeing what it was built to catch.

$$FRR = \frac{FN}{TP + FN}$$

A high FRR in continuous monitoring “might indicate overly stringent thresholds, where normal behavior is often flagged as suspicious, leading to unnecessary alerts and potentially ignoring genuine threats due to alert fatigue.” In practice this shows up as a rule that looks well-behaved on a dashboard — few alerts, low volume — while quietly missing the events it exists to catch, which is exactly why FRR needs to be reviewed on its own rather than inferred from alert count alone.

FAR — False Acceptance Rate

The FAR measures the proportion of benign events incorrectly flagged as threats. A high FAR is the primary driver of alert fatigue: analysts are buried in noise, and real threats get lost in the queue. Unlike FRR, FAR is calculated against the much larger population of benign traffic, so even a small percentage-point increase can translate into hundreds of additional tickets in a busy environment. That volume carries a compounding cost: each false positive an analyst clears trains them, consciously or not, to move faster and scrutinize less on the next one — precisely the condition under which a genuine alert gets closed without a second look.

$$FAR = \frac{FP}{FP + TN}$$

A high FAR means “many potentially harmful activities are not flagged, leaving the organisation vulnerable to undetected threats.” High FAR also desensitises analysts to alerts over time. The two effects reinforce each other: as queue volume grows, average time-per-alert shrinks, and the rule that was supposed to add a layer of scrutiny instead erodes the one already in place.

EER — Equal Error Rate

The EER is the threshold value at which FRR and FAR are equal. It represents the theoretically optimal operating point: the system is equally likely to miss a real threat as to raise a false alarm. The lower the EER, the more accurate the detection system overall, because it means both error types can be held low simultaneously rather than traded off against each other. EER is useful less as a target to hit exactly and more as a benchmark: comparing a rule’s current FRR/FAR position against its own EER shows how much room exists to move the threshold before one error type starts rising faster than the other falls. A rule sitting far from its EER has slack to adjust; a rule already near its EER does not.

$$EER \approx \frac{FRR + FAR}{2}$$

The EER is a critical metric representing the point at which the FRR and FAR are equal. It serves as a balanced measure of the system’s overall accuracy. The lower the EER, the more reliable the system is at distinguishing between legitimate and illegitimate activities.” Framed differently, EER answers a question a single FRR or FAR number cannot: not merely how many errors a rule makes, but how evenly those errors are split between the two directions a detection rule can fail.

Setup

The chunk below defines the core ATLAS formulas, the recommendation logic, and the plotting function used throughout the rest of this report. Calibration charts are built in ggplot2 and passed through `ggplotly()` so readers can hover a curve to read the exact error rate at any threshold, per the project’s visualization stack priority.

Detection Rules

The table below defines the confusion matrix for each detection rule at its current threshold. Edit this cell to analyse a different rule set. Each row represents one detection rule’s classification outcomes over the observation window used for this review period: true positives and false negatives make up the actual-threat population that FRR is measured against, while false positives and true negatives make up the actual-benign population that FAR is measured against. Confusion matrix values should come from a fixed, labeled evaluation window — mixing windows of different lengths or threat densities across rules will skew the FRR/FAR comparison in the summary table below.

► Display code

Table 1: Table 1: Detection rules and confusion matrix values for the current threshold period

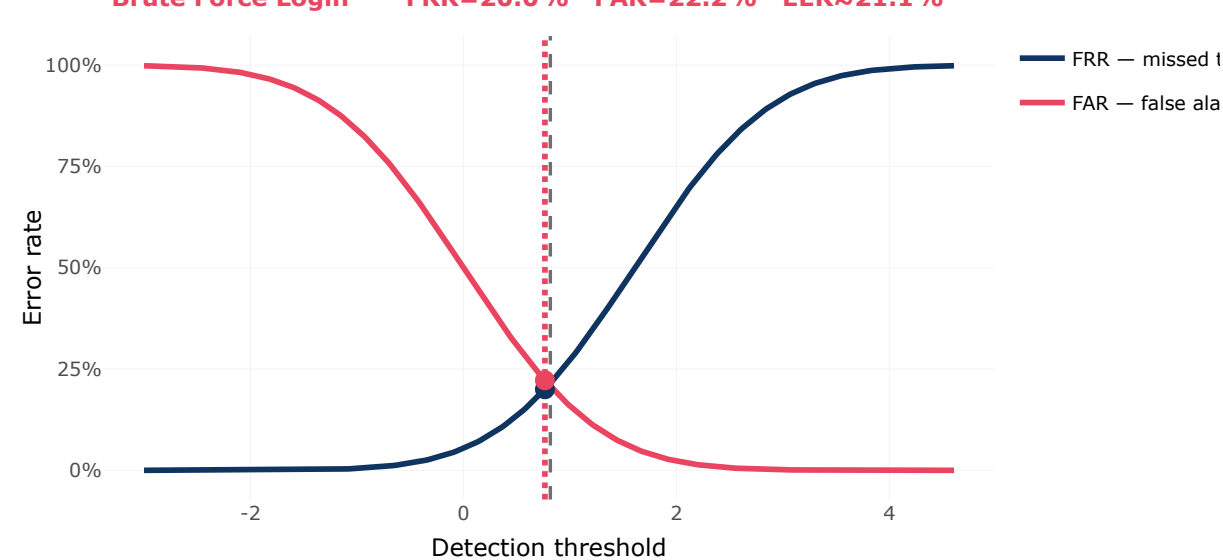
Detection Rule	TP	FP	FN	TN
Brute Force Login	80	200	20	700
Lateral Movement	55	20	45	880
Data Exfiltration	90	90	10	810

Per-Rule Analysis

The three subsections below walk through each rule’s FRR/FAR/EER position in turn, with the calibration chart and plain-language recommendation followed by a short discussion of what the underlying numbers actually mean operationally: why the rule should move, and what moving it will cost or recover.

Brute Force Login

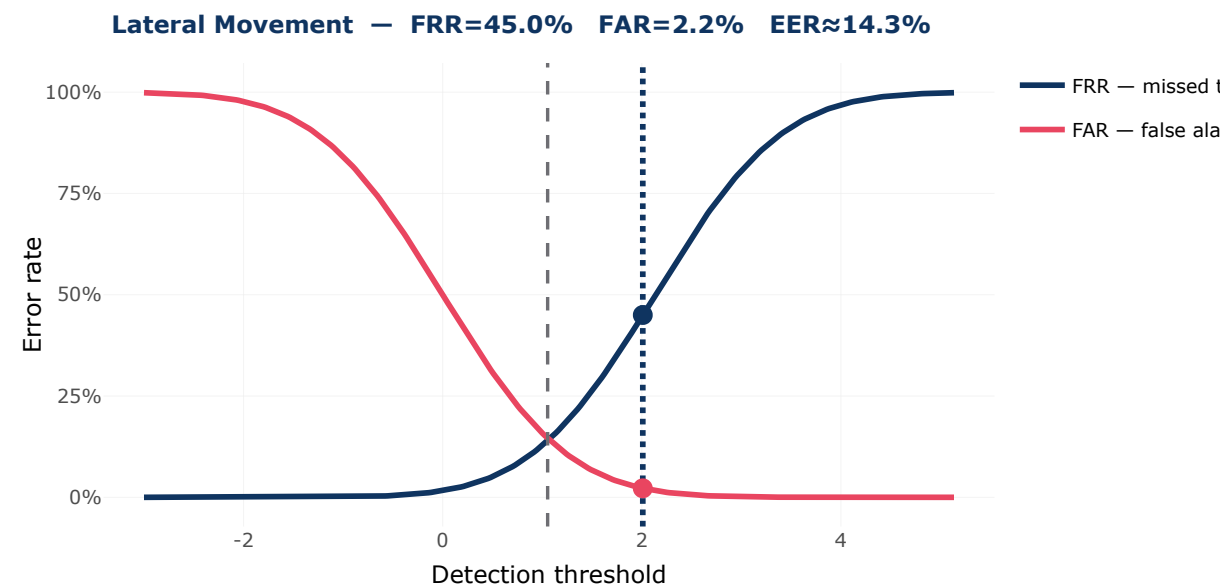
Brute Force Login — FRR=20.0% FAR=22.2% EER≈21.1%



TIGHTEN — FAR (22.2%) > FRR (20.0%) by 2.2%. Alert volume is too high relative to actual threats. Raise the detection threshold to reduce false alarms.

Brute Force Login generates a disproportionate volume of noise: for every real brute-force attempt it catches, roughly 2.5 benign login patterns get flagged alongside it (200 false positives against 80 true positives). That ratio is typical of rules built on a simple failed-attempt counter, which struggles to distinguish a legitimate user who fat-fingers a password three times from an actual credential-stuffing attempt. TIGHTEN here most likely means raising the failed-attempt count or narrowing the retry window before the rule fires, not abandoning the signal — brute-force detection remains a useful control, it just needs less trigger-happy tuning.

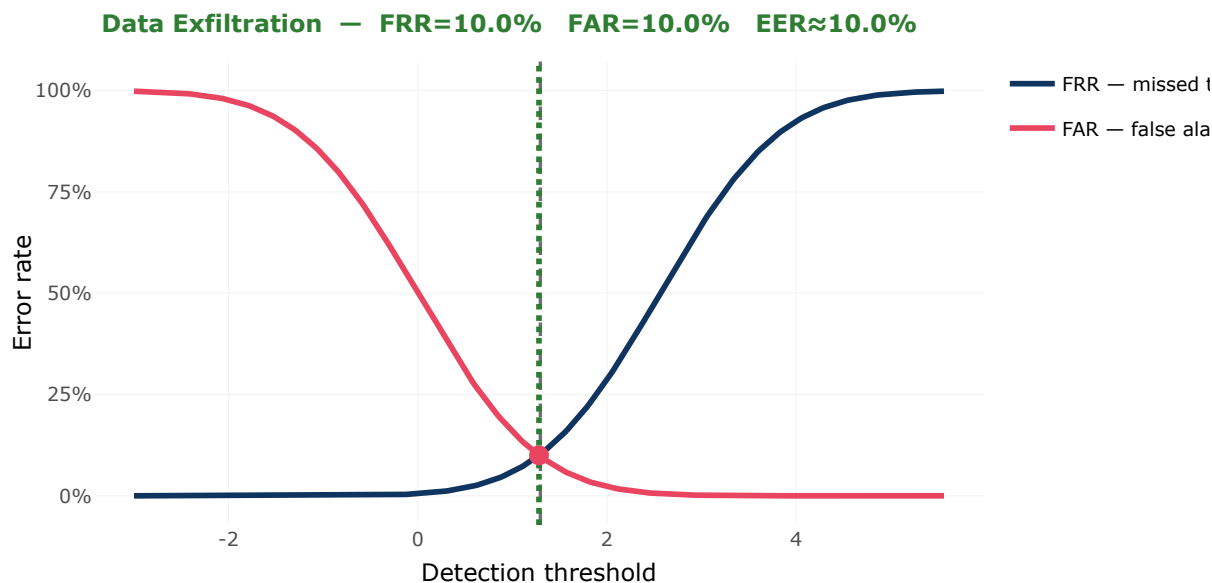
Lateral Movement



LOOSEN — FRR (45.0%) > FAR (2.2%) by 42.8%. Too many real threats are being missed. Lower the detection threshold to increase sensitivity.

Lateral Movement is missing roughly 45.0% of confirmed lateral-movement events at the current threshold — a false-rejection rate that would be unacceptable in almost any other detection category. Its FAR is comparatively very low (2.2%), which is a pattern that often shows up when a rule has been tuned deliberately conservative to avoid flagging legitimate administrative activity such as RDP or PsExec use by IT staff. That conservatism carries a real cost: lateral movement is frequently the step between an initial foothold and the objective of an attack — data theft, ransomware deployment — so a rule that catches only half of it leaves a wide window for an intrusion to progress before containment.

Data Exfiltration



HOLD — FRR (10.0%) and FAR (10.0%) are within 2% of each other. Threshold is near EER — no adjustment needed.

Data Exfiltration is essentially at its EER: FRR and FAR are both 10.0%, meaning the rule is equally likely to miss a real exfiltration event as it is to flag a benign large-file transfer. That balance is the calibration target ATLAS is built around, and for a rule protecting against the highest-consequence outcome in this set — confirmed data loss — sitting at EER rather than skewed toward either error type is the right posture. HOLD does not mean the rule is finished; it means further gains will require a materially better detection signal, such as additional contextual features, rather than a threshold adjustment alone, since moving the threshold in either direction simply trades one error type for the other from this point.

Summary

The table below consolidates the three per-rule results into a single view for cross-rule comparison. Reading FRR and FAR side by side across rules, rather than one rule at a time, is often what surfaces the real prioritization question: Lateral Movement’s 45.0% FRR represents more absolute risk exposure than Brute Force Login’s noisier but individually less consequential 22.2% FAR, even though both rules produce a clear directional recommendation on their own.

Table 2: Table 2: ATLAS threshold analysis results — all detection rules

Detection Rule	FRR	FAR	EER	Recommendation	Rationale
Brute Force Login	20.0%	22.2%	21.1%	TIGHTEN	FAR (22.2%) > FRR (20.0%) by 2.2%. Alert volume is too high relative to actual threats. Raise the detection threshold to reduce false alarms.
Lateral Movement	45.0%	2.2%	14.3%	LOOSEN	FRR (45.0%) > FAR (2.2%) by 42.8%. Too many real threats are being missed. Lower the detection threshold to increase sensitivity.
Data Exfiltration	10.0%	10.0%	10.0%	HOLD	FRR (10.0%) and FAR (10.0%) are within 2% of each other. Threshold is near EER — no adjustment needed.

Interpretation Notes

TIGHTEN rules are generating alert volumes out of proportion to the actual threat rate. The immediate effect of raising the threshold is fewer tickets in the queue — but monitor FRR after adjustment to confirm real threats aren’t being suppressed. Tightening is a one-way lever in the short term: it is easy to raise a threshold and hard to know, without the next Measure cycle’s data, whether the adjustment traded acceptable noise reduction for a genuine increase in missed detections. Stage the change where possible — move the threshold partway, hold for one review period, and confirm FRR is still acceptable before tightening further.

LOOSEN rules have the opposite problem: threats are slipping through. A lower threshold catches more, but will increase analyst workload. Pair loosening with a suppression rule or case-grouping logic where possible to avoid compounding alert fatigue from a different direction. Loosening without a corresponding investment in triage capacity just moves the fatigue problem from one rule to another — the team that was previously drowning in false brute-force alerts will instead start drowning in a wider net of lateral-movement candidates, most of which will still turn out to be benign even after the threshold moves. Budget for the added volume before implementing the change, not after.

HOLD rules are operating near EER. Routine monitoring is sufficient; flag for review if attacker behavior or data volume shifts materially. A rule can drift out of HOLD status without any change to its own threshold — a shift in attacker technique, a new benign traffic pattern introduced by an infrastructure change, or a change in overall event volume can each move a rule's real-world FRR and FAR even while the configured threshold stays fixed. That is why HOLD still calls for the same review cadence as the other two categories, just not the same urgency.

ATLAS Cycle: Next Steps

After implementing threshold adjustments, re-run this report with the confusion matrix values from the next review period. The DataSecure Inc. example in Ch. 8.5.5 demonstrates the expected improvement trajectory: FRR fell from 3% to 0.5%, FAR from 1% to 0.2%, and EER from 2% to 0.35% across one improvement cycle. That trajectory did not happen in a single threshold move — it is the compounding result of repeated Measure → Report → Select cycles, each one narrowing the gap between a rule's actual behavior and its intended purpose. Expect the rules in this report to need more than one cycle to reach a comparable improvement, particularly Lateral Movement, whose current FRR is far enough from target that a single threshold adjustment is unlikely to close the gap on its own.

Why Threshold Calibration Matters to CyberSecurity Leaders

Every number in this report traces back to a decision someone already made and probably doesn't remember making: where to set a threshold. That decision tends to get treated as a technical footnote, owned by whichever engineer configured the rule and never revisited unless something breaks. It shouldn't be. A detection threshold is a risk decision wearing an engineering costume, and the FRR/FAR/EER framework in this report exists to make that decision visible to the people who are actually accountable for the risk it creates.

The threshold is a risk-appetite decision

Every threshold implicitly answers a question the board is already supposed to be asking: how much undetected threat is the organization willing to tolerate, in exchange for how much analyst effort? A rule sitting far to the loose side of its EER, like Lateral Movement in this review, is a de facto statement that the organization accepts a materially higher probability of missing an attacker already inside the network — a statement nobody signed off on, because it was made one configuration change at a time by engineers optimizing for alert volume, not enterprise risk. ATLAS doesn't remove that decision from engineering's hands; it forces the decision to be stated in numbers a risk committee can actually evaluate against a stated risk appetite, rather than left implicit in a threshold field nobody has looked at since the day it was set.

Alert fatigue is a staffing cost with a name

A high FAR shows up on no financial statement, but it has a real cost: analyst hours spent clearing false positives are analyst hours not spent on the alert that matters, and sustained high-FAR conditions are a well-documented driver of security-team attrition. Every point of FAR reduction achieved through calibration, rather than through headcount, is capacity recovered without a hiring requisition. Leaders evaluating a request for additional SOC staff should ask, before approving it, whether the underlying detection rules have been calibrated at all. In several of the cases reviewed here, adjusting a single threshold would recover more analyst hours than the headcount increase being requested to compensate for the noise that threshold is currently generating.

Audit and regulatory defensibility

Under frameworks that expect demonstrable operational resilience and continuous monitoring — DORA's ICT risk management requirements among them — “we have detection rules” is no longer a sufficient answer when a regulator or auditor asks how those rules are validated. A documented, repeatable calibration process, run on a fixed cadence and producing a dated FRR/FAR/EER record for every reviewed rule, is what turns “we monitor for threats” into evidence an auditor can actually test. The absence of that record is itself a finding in an increasing number of regulatory regimes, independent of whether the underlying detection performance is good or bad — a program with mediocre FRR but a documented calibration history is, from an audit standpoint, in better shape than a program with unknown FRR and no history at all.

What this report does not solve

None of this makes threshold calibration free. Loosening a rule to close a coverage gap increases analyst workload immediately and only improves detection if the additional alerts actually get triaged, which requires either idle capacity or a budget conversation — the finding does not execute itself. Tightening a noisy rule can quietly raise FRR if it isn't monitored afterward, trading one blind spot for another instead of removing one. Leaders should treat this report, and the ones that follow it through future ATLAS cycles, as the input to that conversation rather than a substitute for having it. The value of running ATLAS on a fixed cadence is not that it produces a perfect threshold on the first pass — it rarely will — but that it replaces a one-time guess with a record of deliberate, revisited decisions that the organization can stand behind when someone eventually asks why a rule was set the way it was.

Session Information

This report was generated in R; the session details below — R version, platform, and every attached and loaded package — are captured at render time via `sessioninfo::session_info()` for full reproducibility.

```
#> - Session info -----
#>   setting      value
#>   version      R version 4.5.2 (2025-10-31)
#>   os           macOS Tahoe 26.5.1
#>   system       aarch64, darwin20
#>   ui           X11
#>   language     (EN)
#>   collate      en_US.UTF-8
#>   ctype        en_US.UTF-8
#>   tz           America/New_York
#>   date         2026-08-14
#>   pandoc       3.8.3 @ /Applications/RStudio.app/Contents/Resources/app/quarto/bin/tools/aarch64/ (via
rmarkdown)
#>   quarto       1.8.26 @ /usr/local/bin/quarto
#>
#> - Packages -----
#>   package      * version date (UTC) lib source
#>   cli           * 3.6.5   2025-04-23 [1] CRAN (R 4.5.0)
#>   crosstalk     1.2.2   2025-08-26 [1] CRAN (R 4.5.0)
#>   data.table    1.17.8  2025-07-10 [1] CRAN (R 4.5.0)
#>   digest        0.6.39  2025-11-19 [1] CRAN (R 4.5.2)
#>   dplyr         * 1.1.4   2023-11-17 [1] CRAN (R 4.5.0)
#>   evaluate      1.0.5   2025-08-27 [1] CRAN (R 4.5.0)
#>   farver        2.1.2   2024-05-13 [1] CRAN (R 4.5.0)
#>   fastmap       1.2.0   2024-05-15 [1] CRAN (R 4.5.0)
#>   forcats       * 1.0.1   2025-09-25 [1] CRAN (R 4.5.0)
#>   generics      0.1.4   2025-05-09 [1] CRAN (R 4.5.0)
#>   ggplot2       * 4.0.2   2026-02-03 [1] CRAN (R 4.5.2)
#>   glue          1.8.0   2024-09-30 [1] CRAN (R 4.5.0)
#>   gtable        0.3.6   2024-10-25 [1] CRAN (R 4.5.0)
#>   hms           1.1.4   2025-10-17 [1] CRAN (R 4.5.0)
#>   htmltools     0.5.8.1 2024-04-04 [1] CRAN (R 4.5.0)
#>   htmlwidgets   1.6.4   2023-12-06 [1] CRAN (R 4.5.0)
#>   httr          1.4.7   2023-08-15 [1] CRAN (R 4.5.0)
#>   jsonlite      2.0.0   2025-03-27 [1] CRAN (R 4.5.0)
#>   kableExtra    * 1.4.0   2024-01-24 [1] CRAN (R 4.5.0)
#>   knitr         * 1.50    2025-03-16 [1] CRAN (R 4.5.0)
#>   labeling      0.4.3   2023-08-29 [1] CRAN (R 4.5.0)
#>   lazyeval      0.2.2   2019-03-15 [1] CRAN (R 4.5.0)
#>   lifecycle     1.0.5   2026-01-08 [1] CRAN (R 4.5.2)
#>   lubridate     * 1.9.4   2024-12-08 [1] CRAN (R 4.5.0)
#>   magrittr      2.0.4   2025-09-12 [1] CRAN (R 4.5.0)
#>   pillar        1.11.1  2025-09-17 [1] CRAN (R 4.5.0)
#>   pkgconfig     2.0.3   2019-09-22 [1] CRAN (R 4.5.0)
#>   plotly        * 4.11.0  2025-06-19 [1] CRAN (R 4.5.0)
#>   purrr         * 1.2.0   2025-11-04 [1] CRAN (R 4.5.0)
#>   R6            2.6.1   2025-02-15 [1] CRAN (R 4.5.0)
#>   RColorBrewer  1.1-3   2022-04-03 [1] CRAN (R 4.5.0)
#>   readr         * 2.1.5   2024-01-10 [1] CRAN (R 4.5.0)
#>   rlang         1.1.7   2026-01-09 [1] CRAN (R 4.5.2)
#>   rmarkdown     2.30    2025-09-28 [1] CRAN (R 4.5.0)
#>   rstudioapi    0.17.1  2024-10-22 [1] CRAN (R 4.5.0)
#>   S7            0.2.1   2025-11-14 [1] CRAN (R 4.5.2)
#>   scales        * 1.4.0   2025-04-24 [1] CRAN (R 4.5.0)
#>   sessioninfo   * 1.2.3   2025-02-05 [1] CRAN (R 4.5.0)
#>   stringi       1.8.7   2025-03-27 [1] CRAN (R 4.5.0)
#>   stringr       * 1.6.0   2025-11-04 [1] CRAN (R 4.5.0)
#>   svglite       2.2.2   2025-10-21 [1] CRAN (R 4.5.0)
#>   systemfonts   1.3.1   2025-10-01 [1] CRAN (R 4.5.0)
#>   textshaping   1.0.4   2025-10-10 [1] CRAN (R 4.5.0)
#>   tibble        * 3.3.0   2025-06-08 [1] CRAN (R 4.5.0)
#>   tidyr         * 1.3.1   2024-01-24 [1] CRAN (R 4.5.0)
#>   tidyselect    1.2.1   2024-03-11 [1] CRAN (R 4.5.0)
#>   tidyverse     * 2.0.0   2023-02-22 [1] CRAN (R 4.5.0)
#>   timechange     0.3.0   2024-01-18 [1] CRAN (R 4.5.0)
#>   tzdb          0.5.0   2025-03-15 [1] CRAN (R 4.5.0)
#>   vctrs         0.7.1   2026-01-23 [1] CRAN (R 4.5.2)
#>   viridisLite   0.4.3   2026-02-04 [1] CRAN (R 4.5.2)
```

```
#> withr      3.0.2    2024-10-28 [1] CRAN (R 4.5.0)
#> xfun        0.54     2025-10-30 [1] CRAN (R 4.5.0)
#> xml2        1.4.1     2025-10-27 [1] CRAN (R 4.5.0)
#> yaml        2.3.10    2024-07-26 [1] CRAN (R 4.5.0)
#>
#> [1] /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/library
#> * — Packages attached to the search path.
#>
#> _____
```
